



Java: arreglos

Laboratorio — 3º BATIN — Segunda Etapa 2026

Bit, Tech y Gatil organizan la feria estudiantil — y necesitan guardar listas de datos sin escribir cien variables sueltas.

15	16	14	17
0	1	2	3

Objetivo de la clase

1

Entender qué es un arreglo

y por qué reemplaza a tener muchas variables sueltas del mismo tipo.

2

Declarar, acceder y modificar

elementos usando el índice — desde 0 hasta $\text{length} - 1$.

3

Recorrer arreglos con for

aplicando los mismos patrones que ya usaron con ciclos.

4

Resolver problemas típicos

suma, promedio, máximo, mínimo, conteo y búsqueda.

Antes de arrancar: ¿qué venimos usando?

Variables y tipos

```
int, double, boolean, char,  
String
```

Condicionales

```
if / else if / else,  
comparaciones, && || !
```

Ciclos

```
while, do-while, for
```

Hoy agregamos la última pieza grande: **los arreglos**.

Con esto ya tenés todo lo necesario para procesar listas completas de datos del sistema propio (edades, precios, ventas, nombres) antes de guardarlas en la base de datos.

¿Qué es un **arreglo**?

El problema: Bit anota las edades de 4 expositores.

```
int edad1 = 15;  
int edad2 = 16;  
int edad3 = 14;  
int edad4 = 17;
```

¿Y si son 40 expositores? Esto no escala.

La solución: un solo nombre, varios casilleros numerados.

```
int[] edades = {15, 16, 14, 17};
```

15	16	14	17
0	1	2	3

posiciones (índices)

Analogía: es como los casilleros numerados de un kiosco en la feria — todos del mismo tamaño, cada uno con su número fijo, y vos accedés directo al que necesitás por su número, no por su contenido.

Dos formas de declarar un arreglo

1. Ya sabés los valores

```
int[] notas = {8, 5, 9, 7};  
  
String[] puestos =  
    {"Cocina", "Arte"};
```

2. Vas a ir cargando los valores después

```
int[] ventas = new int[5];  
  
// 5 casilleros, todos en 0
```

El tamaño se define una sola vez

Un arreglo de Java tiene un tamaño fijo desde que se crea. No se puede "agregar un casillero más" después — si necesitás más lugar, hay que crear un arreglo nuevo más grande.

```
int[] edades = new int[4]; // tipo[] nombre = new tipo[cantidad];
```

Acceder y modificar un elemento

`arreglo[indice]`

5000	3000	8000
0	1	2

```
int[] precios = {5000, 3000, 8000};  
  
// leer un valor  
System.out.println(precios[1]); // 3000  
  
// modificar un valor  
precios[1] = 4000;
```

⚠ El error más común

El primer elemento está en la posición 0, no en la 1.

5000	3000	8000
0	1	2

Un arreglo de 3 elementos tiene índices válidos 0, 1 y 2 — nunca 3. Usar un índice que no existe produce `ArrayIndexOutOfBoundsException`.

.length: la cantidad de casilleros

15	16	14	17	18
0	1	2	3	4

`edades.length` → 5

`length` te dice cuántos elementos hay — pero el último índice válido es siempre `length - 1`

```
int ultimo = edades[edades.length - 1]; // 18
```

¿Por qué sirve length?

Porque no siempre sabés (o querés escribir a mano) cuántos elementos tiene un arreglo — usar `.length` hace que el código funcione para arreglos de cualquier tamaño, sin tocar nada.

Recorrer un arreglo con for

```
int[] ventas = {2, 5, 3, 8, 1};

for (int i = 0; i < ventas.length; i++) {
    System.out.println(ventas[i]);
}
```

El patrón fijo que siempre se repite

`i` arranca en 0

corta cuando `i < arreglo.length`

avanza de a uno con `i++`

2	5	3	8	1
0	1	2	3	4

i se mueve casillero por casillero (0, 1, 2, 3, 4) hasta recorrer los 5.

Patrón 1: acumulador (suma total)

Bit necesita el total recaudado por su puesto en el día.

```
int[] ventas = {12000, 8000, 15000, 6000};  
int total = 0; // arranca en 0  
  
for (int i = 0; i < ventas.length; i++) {  
    total = total + ventas[i]; // suma cada casillero  
}  
  
System.out.println(total); // 41000
```

Este patrón ya lo conocés

Una variable "acumuladora" arranca en 0, y en cada vuelta del ciclo le sumamos algo — es el mismo patrón que ya usaron para sumar en ejercicios de ciclos, solo que ahora lo que sumamos sale de un arreglo.

Patrón 2: buscar el máximo

Techi quiere saber cuál fue la venta más alta del día.

```
int[] ventas = {12, 30, 8, 45, 20};
int max = ventas[0];          // arranca suponiendo que es el 1º

for (int i = 1; i < ventas.length; i++) { // empieza en 1
    if (ventas[i] > max) {
        max = ventas[i];
    }
}
```

¿Por qué el for empieza en $i = 1$ y no en 0?

Porque ya usamos el elemento 0 para inicializar max — compararlo consigo mismo sería trabajo de más. Para buscar el mínimo es exactamente igual, cambiando $>$ por $<$.

Patrón 3: contar cuántos cumplen algo

Gatil quiere saber cuántos expositores son menores de 18 años.

```
int[] edades = {12, 15, 17, 20, 22, 11, 19};
int menores = 0;

for (int i = 0; i < edades.length; i++) {
    if (edades[i] < 18) {
        menores++;
    }
}
```

12	15	17	20	22	11	19
0	1	2	3	4	5	6

los que cumplen edad < 18 quedan marcados

menores = 4

Arreglos de String y boolean

Mismo mecanismo, cualquier tipo de dato:

```
String[] puestos = {"Robótica", "Reciclaje", "Cocina", "Arte"};  
boolean[] presentes = {true, false, true, true};
```

⚠ Comparar Strings: `.equals()`, no `==`

```
if (nombres[i].equals("Techi")) {
```

En Java, `==` compara si son el mismo objeto en memoria, no si dicen lo mismo. Para texto siempre se usa `.equals()`.

`boolean[]` sirve para listas de sí/no

Ideal para listas de asistencia, checklists, o cualquier dato que sea verdadero/falso por posición — por ejemplo, si cada puesto ya entregó su documentación.

Cargar un arreglo desde el teclado

En vez de escribir los valores en el código, los pedimos con Scanner.

```
Scanner sc = new Scanner(System.in);
int[] edades = new int[4];

for (int i = 0; i < edades.length; i++) {
    System.out.print("Edad del expositor " + i + ": ");
    edades[i] = sc.nextInt();
}
```

Una lectura por vuelta del ciclo

Cada vuelta del for pide un dato con `sc.nextInt()` y lo guarda en el casillero `i`. Así se puede llenar un arreglo de cualquier tamaño con el usuario cargando los datos uno por uno.

El error que todos cometemos alguna vez

```
int[] arr = {1, 2, 3};  
  
// arr tiene 3 elementos:  
// índices válidos: 0, 1, 2  
  
System.out.println(arr[3]);
```

✦ **ArrayIndexOutOfBoundsException**

El programa se detiene: pediste un casillero que no existe.

1	2	3	?
0	1	2	3

Truco para no equivocarse: en los for que recorren todo el arreglo, la condición es siempre `i < arreglo.length` — nunca `<=`.

Los 4 patrones que resuelven casi todo

1

Acumulador

```
total += arreglo[i];
```

sumar, contar total

2

Máximo / mínimo

```
if (arreglo[i] > max)
```

encontrar el mayor o menor

3

Conteo condicional

```
if (condición) contador++;
```

cuántos cumplen algo

4

Búsqueda

```
if (arreglo[i].equals(x))
```

¿está este valor en la lista?

Todos comparten la misma base: declarar una variable antes del for, y actualizarla en cada vuelta según el elemento actual.

Ahora a practicar

Página interactiva: Java — arreglos

- 20 ejercicios de menor a mayor dificultad, con Bit, Tech y Gatil en la feria
- Código editable que se ejecuta de verdad en el navegador — sin internet
- Botón de solución con la explicación de por qué funciona

Próxima clase

Con variables, condicionales, ciclos y arreglos ya en la caja de herramientas, arrancamos con Swing (JFrame, JPanel y componentes) para construir la ventana principal del sistema propio de cada equipo.